

```
# Backward pass & update
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

12. Summary of Benefits

Manual Approach Problem	Dataset + DataLoader Solution
No standard interface	Dataset class with <code>__getitem__</code>
No transformation place	Apply transforms in <code>__getitem__</code>
No shuffling/sampling	<code>shuffle=True</code> and custom samplers
No parallelization	<code>num_workers</code> parameter

Key Takeaway: Always use `Dataset` and `DataLoader` for any real-world PyTorch project. They are essential for efficient, scalable, and clean code.

End of Notes

PyTorch - Building an Artificial Neural Network (ANN) on Fashion-MNIST

Notes for Your Copy

1. Recap of Previous PyTorch Concepts (6 Videos)

Before building the ANN, we covered these fundamentals:

Video	Topic	Key Idea
1	Introduction to PyTorch	Basics, tensors, why PyTorch
2	Tensors	Creating and manipulating tensors
3	Autograd	Automatic differentiation
4	Training Pipeline	Manual training loop
5	<code>nn.Module</code>	Organizing neural networks
6	Dataset & DataLoader	Batching and loading data

Why this matters: These 6 topics are the foundation for building any neural network in PyTorch.

2. Dataset Used: Fashion-MNIST

- **Similar to:** MNIST (handwritten digits) but with clothing items
- **Image size:** 28×28 pixels (784 features per image)
- **Total images:** 70,000
- **Classes:** 10 different clothing types (e.g., T-shirt, trousers, shoes, bag, etc.)
- **Task:** Multi-class classification

Note for this video: We will use only a **subset** of 6,000 images (to train quickly on CPU). Full dataset will be used later with GPU.

3. ANN Architecture for This Task

```
Input Layer:      784 nodes (28×28 flattened)
      ↓
Hidden Layer 1:   128 neurons, Activation = ReLU
      ↓
Hidden Layer 2:   64 neurons, Activation = ReLU
      ↓
Output Layer:     10 neurons, Activation = Softmax (implicit in loss)
```

Why this architecture?

- Input size matches flattened image (784)
 - Hidden layers help learn complex patterns
 - Output layer has 10 neurons – one per class
 - ReLU avoids vanishing gradient problem
 - Softmax gives probability distribution over classes
-

4. Workflow Overview (3 Main Steps)

Step	Description
Step 1	Create DataLoader objects for training and testing data
Step 2	Write training loop (forward pass, loss, backward, update)
Step 3	Write evaluation code (calculate accuracy on test data)

5. Detailed Code with Explanations

5.1 Import Dependencies & Set Seed (Reproducibility)

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import pandas as pd
import numpy as np

# Set seed for reproducibility
torch.manual_seed(42)
```

Explanation: Setting a seed ensures that every time you run the code, you get the same random numbers (same weight initialization, same data splits). This makes your results reproducible.

5.2 Load and Explore the Data

```
# Load dataset (assuming CSV file with 785 columns: label + 784
pixels)
data = pd.read_csv('fashion-mnist.csv')

# First column = label, next 784 columns = pixel values
X = data.iloc[:, 1:].values # features (784 columns)
y = data.iloc[:, 0].values  # labels (0 to 9)
```

Output example (first 16 images visualization):

- Image 1: Shoe (label 9)
 - Image 2: Sandal/shoe (label 7)
 - Image 3: T-shirt (label 0)
 - Image 4: Bag (label 8)
-

5.3 Train-Test Split and Scaling

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Scale pixel values from 0-255 to 0-1 (helps training stability)
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Why scaling? Neural networks train more stably when input values are between 0 and 1 (or -1 and 1). Large values (0-255) can cause exploding gradients.

5.4 Custom Dataset Class

```
class CustomDataset(Dataset):
    def __init__(self, features, labels):
        # Convert to PyTorch tensors with correct data types
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        return self.features[index], self.labels[index]
```

Explanation:

- `__init__`: stores features and labels as tensors
 - Features → `float32` (PyTorch requirement)
 - Labels → `long` (for classification, `CrossEntropyLoss` expects long)
- `__len__`: returns total number of samples
- `__getitem__`: returns one sample (feature + label) at given index

Creating dataset objects:

```
train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)

print(len(train_dataset)) # Output: 4800 (if 6000 total, 80% train)
print(train_dataset[0])  # Output: (tensor of 784 pixels, tensor(7))
```

5.5 Create DataLoaders

```
batch_size = 32

train_loader = DataLoader(train_dataset, batch_size=batch_size,
                           shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
                          shuffle=False)
```

Explanation:

- `batch_size=32`: process 32 images at once
- `shuffle=True` for training: randomizes order each epoch (prevents learning order)
- `shuffle=False` for testing: keep order fixed for consistent evaluation

Number of batches:

```
print(len(train_loader)) # 4800/32 = 150 batches
print(len(test_loader)) # 1200/32 = 38 batches (if 20% test)
```

5.6 Define Neural Network Class

```
class MyNeuralNet(nn.Module):
    def __init__(self, input_features):
        super(MyNeuralNet, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_features, 128), # input → hidden1
            nn.ReLU(),                     # activation
            nn.Linear(128, 64),             # hidden1 → hidden2
            nn.ReLU(),                     # activation
            nn.Linear(64, 10)              # hidden2 → output
            # No softmax here - CrossEntropyLoss applies it internally
        )

    def forward(self, x):
        return self.model(x)
```

Explanation:

- `nn.Sequential`: container that runs layers in order
- `nn.Linear(in, out)`: fully connected layer (weights + bias)
- `nn.ReLU()`: activation function ($\max(0, x)$)
- **No explicit Softmax**: PyTorch's `CrossEntropyLoss` combines `LogSoftmax` + `NLLLoss`

Instantiate model:

```
input_size = X_train.shape[1] # = 784
model = MyNeuralNet(input_size)
```

5.7 Loss Function and Optimizer

```
criterion = nn.CrossEntropyLoss() # for multi-class classification
learning_rate = 0.01
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
```

Explanation:

- **CrossEntropyLoss**: computes negative log likelihood after softmax
 - **SGD (Stochastic Gradient Descent)**: updates weights using gradients
 - `model.parameters()`: all trainable weights and biases in the network
-

5.8 Training Loop

```
epochs = 100
num_batches = len(train_loader)    # = 150

for epoch in range(epochs):
    total_epoch_loss = 0

    for batch_features, batch_labels in train_loader:
        # 1. Forward pass: get predictions
        outputs = model(batch_features)

        # 2. Calculate loss
        loss = criterion(outputs, batch_labels)

        # 3. Clear previous gradients
        optimizer.zero_grad()

        # 4. Backward pass (compute gradients)
        loss.backward()

        # 5. Update weights
        optimizer.step()

        # Accumulate loss for this epoch
        total_epoch_loss += loss.item()

    # Calculate average loss per batch
    avg_loss = total_epoch_loss / num_batches
    print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")
```

Training loop steps (5 steps): | Step | Code | What it does | |-----|-----|-----| | 1 |
outputs = model(batch_features) | Forward pass – get predictions | | 2 | loss =
criterion(outputs, batch_labels) | Compare predictions with true labels | | 3 |
optimizer.zero_grad() | Clear old gradients | | 4 | loss.backward() | Compute new
gradients (backpropagation) | | 5 | optimizer.step() | Update weights using gradients |

Sample output (first few epochs):

```
Epoch 1/100, Loss: 1.3245
Epoch 2/100, Loss: 0.9872
Epoch 3/100, Loss: 0.8456
...
Epoch 100/100, Loss: 0.3124
```

Observation: Loss decreasing → neural network is learning!

5.9 Evaluation Code (Accuracy Calculation)

```
# Set model to evaluation mode
model.eval()

total = 0
correct = 0

# Disable gradient calculation (saves memory & speed)
with torch.no_grad():
    for batch_features, batch_labels in test_loader:
        # Forward pass
        outputs = model(batch_features)

        # Get predicted class (index with highest probability)
        # outputs shape: (batch_size, 10)
        # torch.max returns (values, indices) along dimension 1
        _, predicted = torch.max(outputs, dim=1)

        # Update totals
        total += batch_labels.shape[0]          # add batch size (32)
        correct += (predicted == batch_labels).sum().item()

accuracy = correct / total * 100
print(f"Test Accuracy: {accuracy:.2f}%")
```

Sample output:

```
Test Accuracy: 83.20%
```

Explanation of `torch.max(outputs, dim=1)`:

- `outputs` is a tensor of shape (32, 10) – 32 images, 10 class probabilities each
- `dim=1` means find maximum along the class dimension (columns)
- Returns: (max_values, indices). We only need indices (the predicted class numbers)

Why `model.eval()`?

- Some layers (Dropout, BatchNorm) behave differently during training vs. evaluation
- `eval()` tells the model to switch to evaluation behavior
- Even if not using such layers, it's good practice

Why `torch.no_grad()`?

- Disables gradient tracking
 - Saves memory and speeds up computation
 - Not needed for evaluation since we don't call `.backward()`
-

6. Complete Code Summary (All Together)

```
# 1. Imports
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import pandas as pd
from sklearn.model_selection import train_test_split

# 2. Set seed
torch.manual_seed(42)

# 3. Load data
data = pd.read_csv('fashion-mnist.csv')
X = data.iloc[:, 1:].values
y = data.iloc[:, 0].values

# 4. Train-test split & scaling
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
X_train = X_train / 255.0
X_test = X_test / 255.0

# 5. Custom Dataset
class CustomDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)
    def __len__(self):
        return len(self.features)
    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]

# 6. DataLoaders
train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# 7. Neural Network
class MyNeuralNet(nn.Module):
    def __init__(self, input_features):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(input_features, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
```



```

    )
    def forward(self, x):
        return self.model(x)

# 8. Model, Loss, Optimizer
model = MyNeuralNet(784)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# 9. Training
epochs = 100
for epoch in range(epochs):
    total_loss = 0
    for features, labels in train_loader:
        outputs = model(features)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss:
{total_loss/len(train_loader):.4f}")

# 10. Evaluation
model.eval()
correct = total = 0
with torch.no_grad():
    for features, labels in test_loader:
        outputs = model(features)
        _, predicted = torch.max(outputs, dim=1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
print(f"Accuracy: {100 * correct / total:.2f}%")

```

Final output: Accuracy: 83.20%

7. How to Improve Accuracy (Future Videos)

Technique	Description
Use full dataset (70k images)	More data → better generalization
Train on GPU	Faster training, can use larger models
Different optimizers	Adam, RMSprop (often better than SGD)
Tune learning rate	Find optimal value (e.g., 0.001, 0.01, 0.1)
More epochs	Train longer (but watch for overfitting)

Technique	Description
Weight initialization	Xavier, He initialization
Regularization	L1/L2 regularization
Dropout	Randomly drop neurons during training
Batch normalization	Normalize layer outputs
Hyperparameter tuning	Change hidden layer sizes, number of layers

Goal: Push accuracy from ~83% to >90%

8. Important Notes

1. **Data type matters:** Features → `float32`, Labels → `long`
 2. **No explicit Softmax** when using `CrossEntropyLoss`
 3. **Always call `optimizer.zero_grad()`** before `loss.backward()` (gradients accumulate)
 4. **`model.eval()`** and **`torch.no_grad()`** for evaluation
 5. **Reproducibility:** Set random seeds for consistent results
-

End of Notes. Happy Learning!

PyTorch GPU Training Notes – Fashion MNIST Example

Topic 1: Why Use GPU for Deep Learning?

Explanation (In-depth)

Deep learning models often work with large datasets containing thousands or even millions of images. Training such models on a CPU is very slow because CPUs are designed for general-purpose tasks, not massive parallel computations. GPUs (Graphics Processing Units), on the other hand, have thousands of small cores that can perform many calculations simultaneously. This makes them ideal for the matrix multiplications and tensor operations that form the backbone of neural networks.

In the Fashion MNIST example:

- Total images: 70,000 (60,000 training + 10,000 test)
- When using a CPU, training on the full dataset is too slow, so we often take only a subset (e.g., 10,000 images). This gives lower accuracy (~83%).